

Python 3.14 – Eingebaute Funktionen (Built-in Functions)

Benjamin Malicke

2. Januar 2026

Inhaltsverzeichnis

1	Allgemeine Hinweise	3
2	A – Funktionen, die mit A beginnen	3
2.1	<code>abs(number, /)</code>	3
2.2	<code>aiter(async_iterable, /)</code>	4
2.3	<code>all(iterable, /)</code>	4
2.4	<code>anext(async_iterator, /)</code> <code>anext(async_iterator, default, /)</code>	4
2.5	<code>any(iterable, /)</code>	4
2.6	<code>ascii(object, /)</code>	5
3	B – Funktionen, die mit B beginnen	5
3.1	<code>bin(integer, /)</code>	5
3.2	<code>bool(object=False, /)</code>	5
3.3	<code>breakpoint(*args, **kws)</code>	5
3.4	<code>bytearray(...)</code>	5
3.5	<code>bytes(...)</code>	6
4	C – Funktionen, die mit C beginnen	6
4.1	<code>callable(object, /)</code>	6
4.2	<code>chr(codepoint, /)</code>	6
4.3	<code>@classmethod</code>	6
4.4	<code>compile(source, filename, mode, ...)</code>	7
4.5	<code>complex(...)</code>	7
5	D – Funktionen, die mit D beginnen	7
5.1	<code>delattr(object, name, /)</code>	7
5.2	<code>dict(...)</code>	7
5.3	<code>dir([object])</code>	8
5.4	<code>divmod(a, b, /)</code>	8
6	E – Funktionen, die mit E beginnen	8
6.1	<code>enumerate(iterable, start=0)</code>	8
6.2	<code>eval(source, /, globals=None, locals=None)</code>	8
6.3	<code>exec(source, /, globals=None, locals=None, ...)</code>	8
7	F – Funktionen, die mit F beginnen	8
7.1	<code>filter(function, iterable, /)</code>	8
7.2	<code>float(...)</code>	9
7.3	<code>format(value, format_spec='', /)</code>	9
7.4	<code>frozenset(iterable=(), /)</code>	9

8	G – Funktionen, die mit G beginnen	9
8.1	<code>getattr(object, name, / [, default])</code>	9
8.2	<code>globals()</code>	9
9	H – Funktionen, die mit H beginnen	10
9.1	<code>hasattr(object, name, /)</code>	10
9.2	<code>hash(object, /)</code>	10
9.3	<code>help([object])</code>	10
9.4	<code>hex(integer, /)</code>	10
10	I – Funktionen, die mit I beginnen	10
10.1	<code>id(object, /)</code>	10
10.2	<code>input([prompt])</code>	10
10.3	<code>int(...)</code>	11
10.4	<code>isinstance(object, classinfo, /)</code>	11
10.5	<code>issubclass(class, classinfo, /)</code>	11
10.6	<code>iter(...)</code>	11
11	L – Funktionen, die mit L beginnen	11
11.1	<code>len(object, /)</code>	11
11.2	<code>list(iterable=(), /)</code>	12
11.3	<code>locals()</code>	12
12	M – Funktionen, die mit M beginnen	12
12.1	<code>map(function, iterable, /, *iterables, strict=False)</code>	12
12.2	<code>max(...)</code>	12
12.3	<code>memoryview(object)</code>	12
12.4	<code>min(...)</code>	13
13	N – Funktionen, die mit N beginnen	13
13.1	<code>next(iterator, / [, default])</code>	13
14	O – Funktionen, die mit O beginnen	13
14.1	<code>object()</code>	13
14.2	<code>oct(integer, /)</code>	13
14.3	<code>open(file, mode='r', ...)</code>	13
14.4	<code>ord(character, /)</code>	14
15	P – Funktionen, die mit P beginnen	14
15.1	<code>pow(base, exp, mod=None)</code>	14
15.2	<code>print(*objects, sep=' ', end='\n', ...)</code>	14
15.3	<code>property(fget=None, fset=None, fdel=None, doc=None)</code>	14
16	R – Funktionen, die mit R beginnen	15
16.1	<code>range(...)</code>	15
16.2	<code>repr(object, /)</code>	15
16.3	<code>reversed(object, /)</code>	15
16.4	<code>round(number, ndigits=None)</code>	15
17	S – Funktionen, die mit S beginnen	15
17.1	<code>set(iterable=(), /)</code>	15
17.2	<code>setattr(object, name, value, /)</code>	15
17.3	<code>slice(...)</code>	16
17.4	<code>sorted(iterable, /, *, key=None, reverse=False)</code>	16
17.5	<code>@staticmethod</code>	16

17.6	<code>str(...)</code>	16
17.7	<code>sum(iterable, /, start=0)</code>	16
17.8	<code>super(...)</code>	17
18	T – Funktionen, die mit T beginnen	17
18.1	<code>tuple(iterable=(), /)</code>	17
18.2	<code>type(...)</code>	17
19	V – Funktionen, die mit V beginnen	17
19.1	<code>vars([object])</code>	17
20	Z – Funktionen, die mit Z beginnen	18
20.1	<code>zip(*iterables, strict=False)</code>	18
21	_ – Funktionen, die mit Unterstrich beginnen	18
21.1	<code>__import__(name, globals=None, locals=None, fromlist=(), level=0)</code>	18
22	Zusammenfassung	18

Einleitung

Diese Datei bietet eine deutschsprachige Übersicht über die eingebaute Funktions- und Typenbibliothek von Python 3.14, basierend auf der offiziellen Dokumentation zu den „Built-in Functions“. Für jede eingebaute Funktion oder jeden eingebauten Typ wird eine kurze, verständliche Beschreibung gegeben, ergänzt durch mindestens ein Beispiel in Python-Syntax. Die Funktionssignaturen orientieren sich an der Dokumentation zu Python 3.14.2.

1 Allgemeine Hinweise

- In den Funktionssignaturen markiert der Schrägstrich / positionsgebundene Argumente.
- In den Beispielen wird der interaktive Python-Interpreter mit dem Prompt `>>>` angedeutet.
- Viele eingebaute Namen sind tatsächlich Klassen (z. B. `int`, `list`); sie werden hier der Übersicht halber ebenfalls unter „Funktionen“ geführt.

2 A – Funktionen, die mit A beginnen

2.1 `abs(number, /)`

Beschreibung: Gibt den absoluten Betrag einer Zahl zurück. Das Argument kann ein Integer, ein Gleitkommawert oder ein Objekt sein, das die Methode `__abs__()` implementiert. Für komplexe Zahlen wird der Betrag (die Euklidische Norm) zurückgegeben.

Beispiele:

```
>>> abs(-5)
5
>>> abs(3.14)
3.14
>>> abs(3+4j)
5.0
```

2.2 `aiter(async_iterable, /)`

Beschreibung: Liefert einen asynchronen Iterator für ein asynchron iterierbares Objekt. Entspricht dem Aufruf `x.__aiter__()`. Im Gegensatz zu `iter()` gibt es keine Variante mit zwei Argumenten. Eingeführt in Python 3.10.

Beispiel:

```
async def main(async_iterable):
    async for item in aiter(async_iterable):
        print(item)
```

2.3 `all(iterable, /)`

Beschreibung: Gibt `True` zurück, wenn alle Elemente des Iterables wahr sind (oder das Iterable leer ist). Andernfalls wird `False` zurückgegeben.

Beispiele:

```
>>> all([True, 1, 3])
True
>>> all([True, 0, 3])
False
>>> all([])
True
```

2.4 `anext(async_iterator, /)` `anext(async_iterator, default, /)`

Beschreibung: Asynchrone Variante von `next()`. Beim Awaiting wird das nächste Element des asynchronen Iterators zurückgegeben oder, falls vorhanden, `default`, wenn der Iterator erschöpft ist. Andernfalls wird `StopAsyncIteration` ausgelöst. Eingeführt in Python 3.10.

Beispiele:

```
async def consume(ai):
    try:
        value = await anext(ai)
    except StopAsyncIteration:
        value = None
    return value

async def consume_with_default(ai):
    value = await anext(ai, default=None)
    return value
```

2.5 `any(iterable, /)`

Beschreibung: Gibt `True` zurück, wenn irgendein Element des Iterables wahr ist. Ist das Iterable leer, wird `False` zurückgegeben.

Beispiele:

```
>>> any([0, 0, 1])
True
>>> any([0, 0, 0])
False
>>> any([])
False
```

2.6 `ascii(object, /)`

Beschreibung: Funktioniert ähnlich wie `repr()`, liefert aber eine druckbare Darstellung, in der nicht-ASCII-Zeichen mit `\x-`, `\u-` oder `\U-Escapes` ersetzt werden.

Beispiel:

```
>>> s = "ae-oe-ue"
>>> repr(s)
'"ae-oe-ue"'
>>> ascii(s)
'"\\u00e4\\u00f6\\u00fc"'
```

3 B – Funktionen, die mit B beginnen

3.1 `bin(integer, /)`

Beschreibung: Wandelt eine Ganzzahl in einen binären String mit dem Präfix `0b` um. Für Objekte, die keine Integers sind, muss `__index__()` definiert sein.

Beispiele:

```
>>> bin(3)
'0b11'
>>> bin(-10)
'-0b1010'
```

3.2 `bool(object=False, /)`

Beschreibung: Liefert einen booleschen Wert `True` oder `False`. Das Argument wird gemäß der Standard-Wahrheitswertprüfung konvertiert. Ohne Argument oder mit einem „falschen“ Wert wird `False` zurückgegeben, sonst `True`.

Beispiele:

```
>>> bool(0)
False
>>> bool(1)
True
>>> bool("")
False
>>> bool("Python")
True
```

3.3 `breakpoint(*args, **kws)`

Beschreibung: Einstieg in den Debugger an der aktuellen Stelle. Standardmäßig wird `sys.breakpointhook()` aufgerufen, was meist zu `pdb.set_trace()` führt. Das Verhalten kann über die Umgebungsvariable `PYTHONBREAKPOINT` oder durch Ersetzen von `sys.breakpointhook` angepasst werden.

Beispiel:

```
def foo(x):
    breakpoint() # Debugger-Einstieg
    return x * 2
```

3.4 `bytearray(...)`

Beschreibung: Liefert ein veränderbares Byte-Array-Objekt. Das Argument `source` kann ein String, eine Ganzzahl, ein Objekt mit Buffer-Interface oder ein Iterable von Ganzzahlen im Bereich $0 \leq x < 256$ sein.

Beispiele:

```
>>> bytearray("Hallo", encoding="utf-8")
bytearray(b'Hallo')
>>> bytearray(4)
bytearray(b'\x00\x00\x00\x00')
>>> bytearray([65, 66, 67])
bytearray(b'ABC')
```

3.5 bytes(...)

Beschreibung: Liefert ein unveränderliches Byte-Objekt. Die Argumente entsprechen im Wesentlichen denen von `bytearray()`; dargestellt wird eine Sequenz von Ganzzahlen im Bereich $0 \leq x < 256$.

Beispiele:

```
>>> bytes("Hallo", encoding="utf-8")
b'Hallo'
>>> bytes([65, 66, 67])
b'ABC'
```

4 C – Funktionen, die mit C beginnen

4.1 callable(object, /)

Beschreibung: Prüft, ob ein Objekt „aufrufbar“ erscheint (z. B. Funktionen, Methoden, Klassen). Gibt `True` zurück, wenn ein Aufruf prinzipiell möglich ist, sonst `False`.

Beispiele:

```
>>> callable(len)
True
>>> callable(42)
False

class Foo:
    def __call__(self):
        return "aufgerufen"

>>> f = Foo()
>>> callable(f)
True
```

4.2 chr(codepoint, /)

Beschreibung: Gibt das Zeichen (als String) zu einem gegebenen Unicode-Codepunkt zurück. Umkehrfunktion zu `ord()`.

Beispiele:

```
>>> chr(97)
'a'
>>> chr(8364)
'Euro-Zeichen'
```

4.3 @classmethod

Beschreibung: Dekorator, der eine Methode in eine Klassenmethode verwandelt. Die Methode erhält die Klasse als erstes implizites Argument (oft `cls` genannt), nicht die Instanz.

Beispiel:

```
class Beispiel:
    zahl = 0

    @classmethod
    def set_zahl(cls, value):
        cls.zahl = value

>>> Beispiel.set_zahl(10)
>>> Beispiel.zahl
10
```

4.4 compile(source, filename, mode, ...)

Beschreibung: Übersetzt Quelltext (String oder AST) in ein Codeobjekt, das mit `exec()` oder `eval()` ausgeführt werden kann.

Beispiel:

```
code = compile("x = 1 + 2", "<string>", "exec")
namespace = {}
exec(code, namespace)
>>> namespace["x"]
3
```

4.5 complex(...)

Beschreibung: Erzeugt ein komplexes Zahlobjekt aus Real- und Imaginärteil oder aus einem String.

Beispiele:

```
>>> complex(1, 2)
(1+2j)
>>> complex("1+2j")
(1+2j)
```

5 D – Funktionen, die mit D beginnen

5.1 delattr(object, name, /)

Beschreibung: Löscht ein Attribut mit dem Namen `name` von `object`, sofern erlaubt. Entspricht `del object.name`.

Beispiel:

```
class A:
    def __init__(self):
        self.x = 42

a = A()
delattr(a, "x")
```

5.2 dict(...)

Beschreibung: Erzeugt ein Dictionary-Objekt. Kann mit Schlüsselwortargumenten, einer Mapping-Struktur oder einem Iterable von Schlüssel-Wert-Paaren initialisiert werden.

Beispiele:

```
>>> dict(a=1, b=2)
{'a': 1, 'b': 2}
>>> dict([("a", 1), ("b", 2)])
{'a': 1, 'b': 2}
```

5.3 dir([object])

Beschreibung: Ohne Argument: gibt die Namen im aktuellen lokalen Namensraum zurück. Mit Argument: versucht die gültigen Attribute des Objekts aufzulisten.

Beispiel:

```
>>> dir([])
['__add__', '__class__', ..., 'append', 'clear', 'copy', ...]
```

5.4 divmod(a, b, /)

Beschreibung: Gibt ein Tupel (q, r) zurück, wobei q der ganzzahlige Quotient und r der Rest von a // b bzw. a % b sind.

Beispiele:

```
>>> divmod(7, 3)
(2, 1)
>>> q, r = divmod(20, 6)
>>> q, r
(3, 2)
```

6 E – Funktionen, die mit E beginnen

6.1 enumerate(iterable, start=0)

Beschreibung: Gibt einen Iterator von Tupeln (index, element) zurück, wobei index bei start beginnt.

Beispiele:

```
>>> seasons = ['Spring', 'Summer', 'Fall', 'Winter']
>>> list(enumerate(seasons))
[(0, 'Spring'), (1, 'Summer'), (2, 'Fall'), (3, 'Winter')]
```

6.2 eval(source, /, globals=None, locals=None)

Beschreibung: Wertet einen Ausdruck im gegebenen Namensraum aus und gibt das Ergebnis zurück. Achtung: Ausführung beliebigen Codes, daher nur mit vertrauenswürdigen Eingaben verwenden.

Beispiel:

```
>>> eval("1 + 2 * 3")
7
```

6.3 exec(source, /, globals=None, locals=None, ...)

Beschreibung: Führt einen String oder ein Codeobjekt als Python-Code aus. Anders als eval() wird kein Rückgabewert erwartet.

Beispiel:

```
code = "for i in range(3): print(i)"
exec(code)
```

7 F – Funktionen, die mit F beginnen

7.1 filter(function, iterable, /)

Beschreibung: Konstruiert einen Iterator aus den Elementen von iterable, für die function(item) wahr ist. Ist function None, werden alle „wahren“ Elemente behalten.

Beispiele:


```
>>> list(filter(lambda x: x % 2 == 0, [1, 2, 3, 4]))
[2, 4]
>>> list(filter(None, ["", "a", None, "b"]))
['a', 'b']
```

7.2 float(...)

Beschreibung: Konvertiert ein Argument in eine Gleitkommazahl.

Beispiele:

```
>>> float("1.23")
1.23
>>> float(10)
10.0
```

7.3 format(value, format_spec='', /)

Beschreibung: Wandelt einen Wert gemäß einer Format-Spezifikation in einen formatierten String um. Entspricht `value.__format__(format_spec)`.

Beispiel:

```
>>> format(14, "#b")
'0b1110'
>>> format(3.14159, ".2f")
'3.14'
```

7.4 frozenset(iterable=(), /)

Beschreibung: Erzeugt ein unveränderliches Mengenobjekt (ähnlich `set`, aber nicht veränderbar).

Beispiel:

```
>>> fs = frozenset([1, 2, 2, 3])
>>> fs
frozenset({1, 2, 3})
```

8 G – Funktionen, die mit G beginnen

8.1 getattr(object, name, / [, default])

Beschreibung: Gibt den Wert des Attributs `name` von `object` zurück. Optional kann ein Standardwert angegeben werden, der zurückgegeben wird, falls das Attribut nicht existiert.

Beispiele:

```
class A:
    x = 10

a = A()
>>> getattr(a, "x")
10
>>> getattr(a, "y", None)
None
```

8.2 globals()

Beschreibung: Gibt das Dictionary des aktuellen Modul-Namensraums zurück.

Beispiel:

```
>>> "globals" in globals()
True
```

9 H – Funktionen, die mit H beginnen

9.1 `hasattr(object, name, /)`

Beschreibung: Gibt True zurück, wenn das Objekt ein Attribut mit dem Namen `name` hat.

Beispiel:

```
>>> hasattr([], "append")
True
```

9.2 `hash(object, /)`

Beschreibung: Gibt den Hashwert eines Objekts zurück, sofern definiert. Wird für Dictionary-Schlüssel und Mengen verwendet.

Beispiel:

```
>>> hash("abc") == hash("abc")
True
```

9.3 `help([object])`

Beschreibung: Startet das eingebaute Hilfesystem oder zeigt Hilfe zu einem Objekt, einer Funktion oder einem Modul an.

Beispiel:

```
>>> help(len)
```

9.4 `hex(integer, /)`

Beschreibung: Wandelt eine Ganzzahl in einen hexadezimalen String mit Präfix `0x` um.

Beispiele:

```
>>> hex(255)
'0xff'
>>> hex(-42)
'-0x2a'
```

10 I – Funktionen, die mit I beginnen

10.1 `id(object, /)`

Beschreibung: Gibt eine eindeutige Identität eines Objekts für dessen Lebensdauer zurück (in CPython meist die Speicheradresse).

Beispiel:

```
>>> id(1) == id(1)
True
```

10.2 `input([prompt])`

Beschreibung: Liest eine Zeile von der Standardeingabe und gibt sie als String zurück. Optional kann ein Prompt angezeigt werden.

Beispiel:

```
name = input("Wie heisst du? ")
print("Hallo,", name)
```

10.3 int(...)

Beschreibung: Konvertiert ein Argument in eine Ganzzahl oder gibt 0 zurück, wenn kein Argument übergeben wird. Strings können in verschiedenen Basiszahlen interpretiert werden.

Beispiele:

```
>>> int(123.45)
123
>>> int("123")
123
>>> int("FACE", 16)
64206
```

10.4 isinstance(object, classinfo, /)

Beschreibung: Prüft, ob object eine Instanz von classinfo oder einer Unterklasse davon ist. classinfo kann ein Typ oder ein Tupel von Typen sein.

Beispiele:

```
>>> isinstance(3, int)
True
>>> isinstance(True, (int, bool))
True
```

10.5 issubclass(class, classinfo, /)

Beschreibung: Prüft, ob class eine Unterklasse von classinfo ist. classinfo kann ein Typ oder ein Tupel von Typen sein.

Beispiel:

```
>>> issubclass(bool, int)
True
```

10.6 iter(...)

Beschreibung: Gibt einen Iterator zurück. Ohne zweites Argument muss das Objekt iterierbar sein; mit zweitem Argument (sentinel) wird iter(callable, sentinel) verwendet.

Beispiele:

```
>>> it = iter([1, 2, 3])
>>> next(it)
1
```

```
from functools import partial
with open("data.bin", "rb") as f:
    for block in iter(partial(f.read, 64), b''):
        process(block)
```

11 L – Funktionen, die mit L beginnen

11.1 len(object, /)

Beschreibung: Gibt die Länge (Anzahl der Elemente) eines Objekts zurück, z. B. einer Sequenz oder Sammlung.

Beispiele:

```
>>> len("Hallo")
5
>>> len([1, 2, 3])
3
```

11.2 list(iterable=(), /)

Beschreibung: Erzeugt eine neue Liste aus einem Iterable.

Beispiele:

```
>>> list("abc")
['a', 'b', 'c']
>>> list((1, 2, 3))
[1, 2, 3]
```

11.3 locals()

Beschreibung: Gibt eine Mapping-Repräsentation der aktuellen lokalen Symboltabelle zurück. Verhalten hängt vom Kontext (Modul, Funktion, Klasse) ab.

Beispiel:

```
def f():
    x = 42
    return locals()

>>> f()
{'x': 42}
```

12 M – Funktionen, die mit M beginnen

12.1 map(function, iterable, /, *iterables, strict=False)

Beschreibung: Wendet function auf jedes Element der gegebenen Iterable(s) an und liefert einen Iterator mit den Ergebnissen.

Beispiele:

```
>>> list(map(lambda x: x * 2, [1, 2, 3]))
[2, 4, 6]
>>> list(map(pow, [1, 2, 3], [2, 3, 4]))
[1, 8, 81]
```

12.2 max(...)

Beschreibung: Gibt das größte Element eines Iterables oder das größte der übergebenen Argumente zurück. Optional mit key-Funktion und default-Wert.

Beispiele:

```
>>> max([1, 5, 2])
5
>>> max(["a", "bbb", "cc"], key=len)
'bbb'
```

12.3 memoryview(object)

Beschreibung: Erzeugt ein memoryview-Objekt, das direkten Zugriff auf den Puffer eines Objekts (z.B. bytes) erlaubt, ohne zu kopieren.

Beispiel:

```
>>> m = memoryview(b"ABC")
>>> m[0]
65
```

12.4 min(...)

Beschreibung: Analog zu `max()`, aber für das kleinste Element.

Beispiele:

```
>>> min([1, 5, 2])
1
>>> min(["a", "bbb", "cc"], key=len)
'a'
```

13 N – Funktionen, die mit N beginnen

13.1 next(iterator, / [, default])

Beschreibung: Ruft `iterator.__next__()` auf und gibt das nächste Element zurück. Optional kann ein Standardwert angegeben werden, der bei erschöpftem Iterator zurückgegeben wird.

Beispiele:

```
>>> it = iter([1, 2])
>>> next(it)
1
>>> next(it)
2
>>> next(it, None)
None
```

14 O – Funktionen, die mit O beginnen

14.1 object()

Beschreibung: Gibt eine neue, leere Objektinstanz zurück. Oberste Basisklasse aller neuen Klassen.

Beispiel:

```
>>> o = object()
>>> isinstance(o, object)
True
```

14.2 oct(integer, /)

Beschreibung: Wandelt eine Ganzzahl in einen oktalen String mit Präfix `0o` um.

Beispiele:

```
>>> oct(8)
'0o10'
>>> oct(-56)
'-0o70'
```

14.3 open(file, mode='r', ...)

Beschreibung: Öffnet eine Datei und gibt ein Dateiojekt zurück. Der Modus bestimmt, ob gelesen, geschrieben oder angehängt wird und ob im Text- oder Binärmodus gearbeitet wird.

Beispiel:

```
with open("example.txt", "w", encoding="utf-8") as f:
    f.write("Hallo Welt\n")
```

14.4 ord(character, /)

Beschreibung: Umkehrfunktion zu chr(). Gibt den Unicode-Codepunkt eines einzelnen Zeichens (oder den Wert eines einzelnen Bytes in bytes/bytearray) zurück.

Beispiele:

```
>>> ord('a')
97
>>> ord('Euro-Zeichen')
8364
```

15 P – Funktionen, die mit P beginnen

15.1 pow(base, exp, mod=None)

Beschreibung: Berechnet base ** exp bzw. pow(base, exp, mod) effizient als Potenz mit Modulo. Für Integers unterstützt die dreistellige Variante auch modularen Inversen.

Beispiele:

```
>>> pow(2, 10)
1024
>>> pow(2, 10, 1000)
24
```

15.2 print(*objects, sep=' ', end='\n', ...)

Beschreibung: Gibt die Objekte als Text auf einen Ausgabestrom aus, getrennt durch sep und abgeschlossen durch end.

Beispiele:

```
>>> print("A", "B", "C", sep=" ", end=" ")
A, B, C
>>> print("Keine neue Zeile", end="")
Keine neue Zeile>>>
```

15.3 property(fget=None, fset=None, fdel=None, doc=None)

Beschreibung: Erzeugt ein Property-Objekt zur Kapselung von Attributzugriff (Getter/Setter/Deleter).

Beispiel:

```
class C:
    def __init__(self):
        self._x = None

    @property
    def x(self):
        "Ich bin das 'x'-Property."
        return self._x

    @x.setter
    def x(self, value):
        self._x = value

c = C()
c.x = 10
>>> c.x
10
```

16 R – Funktionen, die mit R beginnen

16.1 range(...)

Beschreibung: Liefert ein unveränderliches Sequenzobjekt für Ganzzahlbereiche, meist verwendet in Schleifen.

Beispiele:

```
>>> list(range(3))
[0, 1, 2]
>>> list(range(1, 5, 2))
[1, 3]
```

16.2 repr(object, /)

Beschreibung: Gibt eine möglichst eindeutige Stringrepräsentation eines Objekts zurück, die im Idealfall mit `eval()` wieder ein gleichwertiges Objekt erzeugen kann.

Beispiel:

```
>>> repr("Hallo")
"'Hallo'"
```

16.3 reversed(object, /)

Beschreibung: Gibt einen Iterator zurück, der eine Sequenz in umgekehrter Reihenfolge durchläuft.

Beispiel:

```
>>> list(reversed([1, 2, 3]))
[3, 2, 1]
```

16.4 round(number, ndigits=None)

Beschreibung: Rundet eine Zahl auf `ndigits` Nachkommastellen. Bei `None` wird auf die nächste Ganzzahl gerundet (Banker's Rounding).

Beispiele:

```
>>> round(2.675, 2)
2.67
>>> round(1.5)
2
```

17 S – Funktionen, die mit S beginnen

17.1 set(iterable=(), /)

Beschreibung: Erzeugt ein veränderbares Mengenobjekt aus einem Iterable.

Beispiele:

```
>>> set([1, 2, 2, 3])
{1, 2, 3}
```

17.2 setattr(object, name, value, /)

Beschreibung: Setzt das Attribut `name` eines Objekts auf `value`. Entspricht `object.name = value`.

Beispiel:

```
class A:
    pass

a = A()
setattr(a, "x", 10)
>>> a.x
10
```

17.3 slice(...)

Beschreibung: Erzeugt ein Slice-Objekt, das insbesondere bei erweiterter Indexierung verwendet wird.

Beispiel:

```
s = slice(1, 5, 2)
>>> [0, 1, 2, 3, 4, 5][s]
[1, 3]
```

17.4 sorted(iterable, /, *, key=None, reverse=False)

Beschreibung: Gibt eine neue, sortierte Liste aus den Elementen des Iterables zurück.

Beispiele:

```
>>> sorted([3, 1, 2])
[1, 2, 3]
>>> sorted(["a", "bbb", "cc"], key=len, reverse=True)
['bbb', 'cc', 'a']
```

17.5 @staticmethod

Beschreibung: Dekorator zur Definition einer statischen Methode, die weder `self` noch `cls` implizit erhält.

Beispiel:

```
class C:
    @staticmethod
    def add(a, b):
        return a + b

>>> C.add(1, 2)
3
```

17.6 str(...)

Beschreibung: Eingebaute String-Klasse und Konvertierungsfunktion zu Text.

Beispiele:

```
>>> str(123)
'123'
>>> str(b"abc", encoding="utf-8")
'abc'
```

17.7 sum(iterable, /, start=0)

Beschreibung: Summiert die Elemente eines Iterables von links nach rechts, beginnend mit `start`.

Beispiele:


```
>>> sum([1, 2, 3])
6
>>> sum([1, 2, 3], start=10)
16
```

17.8 super(...)

Beschreibung: Liefert ein Proxy-Objekt, das Methodenaufrufe an Basisklassen delegiert. Wird häufig für kooperatives Mehrfacherben verwendet.

Beispiel:

```
class A:
    def greet(self):
        print("Hallo von A")

class B(A):
    def greet(self):
        super().greet()
        print("... und B")

>>> B().greet()
Hallo von A
... und B
```

18 T – Funktionen, die mit T beginnen

18.1 tuple(iterable=(), /)

Beschreibung: Erzeugt ein unveränderliches Tupel aus einem Iterable.

Beispiele:

```
>>> tuple([1, 2, 3])
(1, 2, 3)
```

18.2 type(...)

Beschreibung: Mit einem Argument: gibt den Typ eines Objekts zurück. Mit drei Argumenten: erzeugt einen neuen Typ (eine Klasse) dynamisch.

Beispiele:

```
>>> type(1)
<class 'int'>

X = type("X", (), {"a": 1})
>>> X.a
1
```

19 V – Funktionen, die mit V beginnen

19.1 vars([object])

Beschreibung: Gibt das `__dict__`-Attribut eines Objekts zurück, wenn vorhanden. Ohne Argument verhält es sich ähnlich wie `locals()`.

Beispiel:

```
class A:
    def __init__(self):
        self.x = 1
```

```
self.y = 2
```

```
a = A()
>>> vars(a)
{'x': 1, 'y': 2}
```

20 Z – Funktionen, die mit Z beginnen

20.1 zip(*iterables, strict=False)

Beschreibung: Iteriert parallel über mehrere Iterables und erzeugt Tupel der entsprechenden Elemente. Standardmäßig stoppt `zip` bei der kürzesten Sequenz.

Beispiele:

```
>>> list(zip([1, 2, 3], ['a', 'b', 'c']))
[(1, 'a'), (2, 'b'), (3, 'c')]

x = [1, 2, 3]
y = [4, 5, 6]
>>> x2, y2 = zip(*zip(x, y))
>>> list(x2), list(y2)
([1, 2, 3], [4, 5, 6])
```

21 _ – Funktionen, die mit Unterstrich beginnen

21.1 __import__(name, globals=None, locals=None, fromlist=(), level=0)

Beschreibung: Fortgeschrittene Import-Funktion, die vom `import`-Statement aufgerufen wird. In normalem Code sollte stattdessen `import` bzw. `importlib.import_module()` verwendet werden.

Beispiele:

```
>>> m = __import__("math")
>>> m.sqrt(16)
4.0

# Entspricht in etwa: from math import sqrt
>>> _temp = __import__("math", globals(), locals(), ["sqrt"], 0)
>>> _temp.sqrt(9)
3.0
```

22 Zusammenfassung

Diese Übersicht fasst die eingebauten Funktionen und Typen von Python 3.14 in deutscher Sprache zusammen und illustriert ihre Verwendung mit kurzen Codebeispielen. Sie ist als praxisorientierte Ergänzung zur offiziellen Python-Dokumentation gedacht und soll beim Lernen, Nachschlagen und Unterrichten unterstützen.